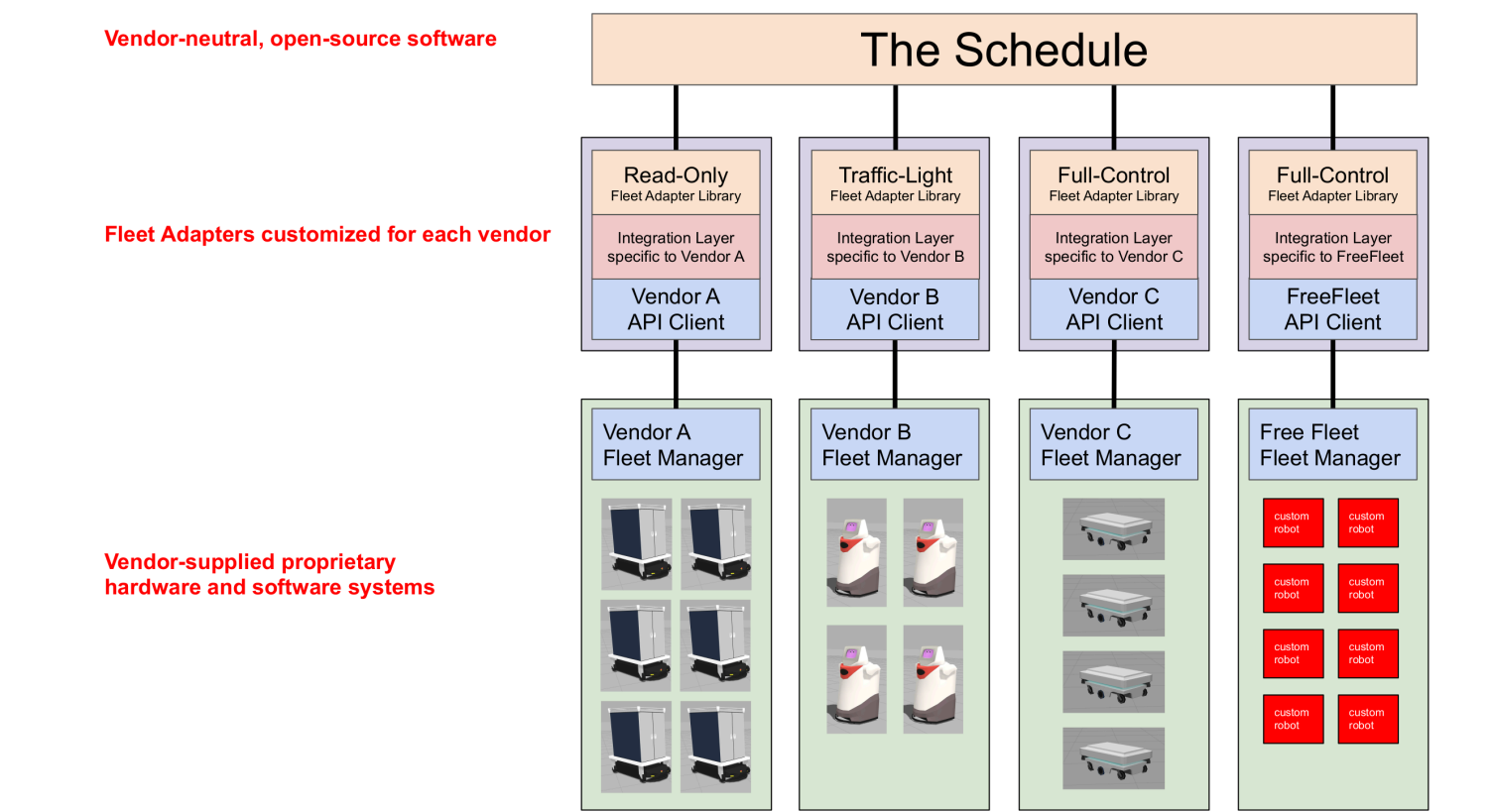




Fleet Adapters

Each robot fleet that participates in an RMF deployment is expected to have a fleet adapter that connects its fleet-specific API to the interfaces of the core RMF traffic scheduling and negotiation system. The fleet adapter is also responsible for handling communication between the fleet and the various standardized smart infrastructure interfaces, e.g. to open doors, summon lifts, and wake up dispensers.

Different robot fleets have different features and capabilities, dependent on how they were designed and developed. The traffic scheduling and negotiation system does not postulate assumptions about what the capabilities of the fleets will be. However, to minimize the duplication of integration effort, we have identified 4 different broad categories of control that we expect to encounter among various real-world fleet managers.

Fleet adapter type	Robot/Fleetmanager API feature set	Remarks
Full Control	Read the current location of the robot [x,	RMF is provided with live status updates and full control over the paths that each individual mobile

Fleet adapter type	Robot/Fleetmanager API feature set	Remarks
	y, yaw] • Request robot to move to [x, y, yaw] coordinate • Pause a robot while it is navigating to [x, y, yaw] • Resume a paused robot • Get route/path taken by robot to destination • ETA to destination • Read battery status of the robot • Infer when robot is done navigating to [x, y, yaw] • Send robot to docking/charging station • Switch on board map and re-localize robot. • Start a process (such as clean Zone_A) • Pause/resume/stop process • Infer when process is complete (specific to use case)	robot uses when navigating through the environment. This control level provides the highest overall efficiency and compliance with RMF, which allows RMF to minimize stoppages and deal with unexpected scenarios gracefully. <i>(API available)</i>
Traffic Light	• Read the current location of the robot [x, y, yaw] • Pause a robot while it is navigating to [x, y, yaw] • Resume a paused robot	RMF is given the status as well as pause/resume control over each mobile robot, which is useful for deconflicting traffic schedules especially when sharing resources like corridors, lifts and doors. <i>*(API available)</i>

Fleet adapter type	Robot/Fleetmanager API feature set	Remarks
	• Read battery status of the robot • Send robot to docking/charging station • Start a process (such as clean Zone_A) • Pause/resume/stop process • Infer when process is complete (specific to use case)	
Read Only	• Read the current location of the robot [x, y, yaw] • Read or infer the path that the robot will take to its current destination • Read average speed of the robot or ETA to destination • Read battery status of the robot • Infer when process is complete (specific to use case)	RMF is not given any control over the mobile robots but is provided with regular status updates. This will allow other mobile robot fleets with higher control levels to avoid conflicts with this fleet. <i>Note that any shared space is allowed to have a maximum of just one "Read Only" fleet in operation. Having none is ideal. (Preliminary API available)</i>
No Interface		Without any interface to the fleet, other fleets cannot coordinate with it through RMF, and will likely result in deadlocks when sharing the same navigable environment or resource. This level will not function with an RMF-enabled environment. <i>(Not compatible)</i>

In short, the more collaborative a fleet is with RMF, the more harmoniously all of the fleets and systems are able to operate together. Note again that there can only ever be one "Read Only" fleet in a shared space, as any two or more of such fleets will make avoiding deadlock or resource conflict nearly impossible.

Currently we provide a reusable C++ API (as well as Python bindings) for integrating the **Full Control** category of fleet management. A preliminary ROS 2 message API is available for the **Read Only** category, but that API will be deprecated in favor of a C++ API (with [Python bindings](#) available) in a future release. The **Traffic Light** control category is compatible with the core RMF scheduling system, but we have not yet implemented a reusable API for it. To implement a **Traffic Light** fleet adapter, a system integrator would have to use the core traffic schedule and negotiation APIs directly, as well as implement the integration with the various infrastructure APIs (e.g. doors, lifts, and dispensers).

The API for the **Full Control** category is described in the [Mobile Robot Fleets](#) section of the Integration chapter, and the **Read Only** category is described in the [Read Only Fleets](#) section of the Integration chapter.

Found a bug? [Edit this page on GitHub](#).

Frequently Asked Questions

Why is this traffic management system so complicated?

RMF has a number of system design constraints that create unique challenges for traffic management. The core goal of RMF is to facilitate system integration for heterogeneous mobile robot fleets that may be provided by different vendors and may have different technical capabilities.

Vendors tend to want to keep their computing systems independent from other vendors. Since vendors are often responsible for ensuring uptime and reliability on their computing infrastructure, they may view it as an unacceptable liability to share computing resources with another vendor. This means that the traffic management system must be able to function while being distributed across different machines on a network.

Different robot platforms may have different capabilities. Many valuable AGV platforms that are currently deployed are not able to change their itineraries dynamically. Some AGV platforms can change course when instructed to, as long as they stick to a predefined navigation graph. Some AMR platforms can dynamically navigate themselves around unanticipated obstacles in their environment. Since RMF is meant to be an enabling technology, it is important that we design a system that can maximize the utility of all these different types of systems without placing detrimental constraints on any of them.

These considerations led to the current design of distributed conflict prevention and distributed schedule negotiation. There is plenty of space within the design to create simpler and more efficient subsets for categories of mobile robots that fit certain sets of requirements, but these optimizations can be added later, building on top of the existing completely generalized framework.

Who opens and closes doors and operates the lifts? The robot or RMF? Or both?

The responsibility of knowing when a door needs to be opened and then sending the command to open it belongs to the "fleet adapter". The basic design is:

- The fleet adapter keeps track of the robot's progress
- When the robot needs to go through a door, the fleet adapter will recognize this
- The fleet adapter will send a signal to the door to open
- Once the door is open, the fleet adapter will command the robot to proceed

- Once the robot is through the door, the fleet adapter will command the robot wait until the door is closed
- The fleet adapter will command the door to close
- Once the door is closed, the fleet adapter will command the robot to proceed

The way a fleet adapter knows about the doors is by parsing the navigation graph that is provided to it. The navigation graph is a required parameter for the `full_control` type of fleet adapter. `rmf_demos` shows an example of providing a navigation graph to the fleet adapter.

The recommended way to construct a navigation graph is to use the `traffic-editor` tool. The `rmf_demos` repos shows some examples of `traffic-editor` project files.

However, it's entirely possible to construct your own navigation graphs. They use YAML format.

Are lifts supported?

Proper lift support (meaning, specifying an actual lift that can move between floors, and exporting that information into the navigation graph) is not something that has been developed yet.

However, for testing and demonstration purposes, there are two special navigation graph edge properties that can allow a RMF fleet adapter to emulate lift usage. This is meant for demo scenarios where a "mock lift" has been created that receives lift commands and transmits lift states but does not actually move between any different floors in a building. For example, tape on the floor of a lab to indicate the "lift cabin" box, to allow development and testing without occupying the actual building lift.

These properties were initially included for demonstration purposes, but they are proving useful enough that we might make them officially supported properties. Due to the cost and scarcity of "real" lifts, there seems to be broad interest in having single-floor hardware test setups that emulate multi-floor scenarios.

The edge properties are:

- `demo_mock_floor_name` : The name of the floor that the robot is on while traversing the edge
- `demo_mock_lift_name` : The name of the lift that is being entered or exited while the robot traverses the edge

The idea is that if you have a single floor demonstration environment but want to demonstrate interaction with a lift, then you can set up a mock "lift" and imagine that each side of the "lift" opens to a different floor, and the robot is only allowed to enter/exit that side of the "lift" when the "lift" believes it is on that floor. This emulates lift cabins with two sets of doors.

To make this idea more concrete, imagine you have a single-floor hardware testing area, and a box is drawn on the ground with an LED display next to it that reads off pretend floor names. The mock lift will transmit lift state messages that match up with whatever floor the LED is displaying. There is also some indication of whether the lift doors are open or closed. You can further imagine that entering or exiting from west side of the "lift" is only allowed when the lift believes it is on floor L1 whereas entering or exiting the "lift" from the east side is only allowed when it believes it is on floor L3.

In that setup, for a robot to "correctly" navigate from a waypoint on L1 to a waypoint on L3, the robot needs to:

- Approach the "lift" from the west side
- Call the "lift" down to L1
- Wait until the lift state has it on floor L1 with the doors open
- Move into the "lift" (i.e. the box drawn on the ground) and request that it "moves" to L3
- Wait until the "lift" indicates that it has reached L3 and that its doors are open
- Exit the "lift" on the east side

A rough ASCII diagram would look like this (numbers are waypoints and letters are edges):

```
1 <---a---> 2 <---b---> 3
```

- Waypoint 1 is on floor L1
- Waypoint 2 is inside the "lift" named LIFT001
- Waypoint 3 is on floor L3
- The properties of edge `a` are:
 - `bidirectional: true`
 - `demo_mock_floor_name: L1`
 - `demo_mock_lift_name: LIFT001`
- The properties of edge `b` are:
 - `bidirectional: true`
 - `demo_mock_floor_name: L3`
 - `demo_mock_lift_name: LIFT001`

If multiple fleets can do the same task, which one is chosen?

Though not implemented yet, there is a design worked out for a bidding system where a task request will be converted to a bid request. The bid request will be sent to each fleet adapter, and each fleet adapter that can perform the task will report its best estimate for how soon it would be able to have the task finished. The fleet adapter that offers the lowest bid will be assigned the task.

The API and implementation are awaiting finalization of some critical components.

Can some robots have priority over other robots?

The negotiation system concept does support prioritization for which robot will accommodate the other robot(s). Any arbitrary metric or weighting system can be used when resolving a negotiation. But in the current implementation that we are using, we treat all vehicles as equal and choose the resolution that minimizes the net delay across all the robots, without any prioritization or weighting.

Since this codebase is open source, you can easily fork the code and modify it to use any prioritization system that you'd like. Specifically, replace

```
rmf_traffic::schedule::QuickestFinishEvaluator() with your own  
Negotiation::Evaluator class that behaves in whatever way you would like.
```

What distance is maintained between two robots?

This is configurable. There are two relevant parameters: `footprint_radius` and `vicinity_radius`. The `footprint_radius` represents an estimate of the vehicle's physical footprint. The `vicinity_radius` represents an estimate of the region which the robot needs other vehicles to stay clear of. A "schedule conflict" is defined as an instance where one vehicle's "footprint" is scheduled to enter another vehicle's "vicinity". The job of the negotiation system is to come up with a fix to the schedule that keeps all vehicles' "footprints" out of all other vehicles' "vicinities".

How is job dispatching accomplished?

The Dispatch Planner module is currently a work-in-progress. So far, the demo platforms that have been developed for RMF did not require a planner to dispatch tasks to different fleets, because in all demo platforms done thus far (late 2020), each task type could only be performed by one (and only one) robot fleet. So the task dispatching has been trivial: the task gets assigned to whatever fleet is capable of performing the task, while the rest of the fleets just ignore the task request. We are currently working on a true dispatch planner and formal task bidding system which we're aiming to include in release 1.2.0, slated for end of December 2020. The idea is that every fleet that can perform a task request will offer a bid for how much it would "cost" them to perform a task, and the bid with the lowest "cost" will be the winner. The "cost" will be determined by two factors:

- How quickly the task is finished

- How much other tasks get delayed if the new task needs to preempt them

What is `rmf_traffic` ?

`rmf_traffic` provides a middleware-neutral implementation of the core traffic scheduling algorithms and utilities. It does not use or depend on ROS 2.

What is `rmf_traffic_ros2` ?

`rmf_traffic_ros2` provides convenient wrappers for using the `rmf_traffic` as part of a distributed ROS 2 system.

Where is the costmap?

There is no costmap representation in `rmf_core`. Costmaps are one method that is typically used for representing volume occupancy for autonomous navigation planning. While it's true that the traffic utilities deal with navigation planning, they are principally concerned about identifying conflicts between the intended routes of autonomous vehicles. The traffic utilities are not responsible for the "local" navigation of vehicles around static obstacles in their environment.

Different robot platforms often have different representations of costmaps or other navigation algorithms, many of which may be proprietary. On top of that, different robot platforms with the same costmap representations may still require different costmap values because of differences in robot footprints. Because of these factors, we leave it to the robot platforms themselves to determine how they represent their costmaps. If it's important for a system integrator to take a robot's costmap into account when performing traffic planning, they can implement a custom `rmf_traffic::agv::RouteValidator` that uses the robot's custom costmap when determining whether a candidate route is valid.

What is the core algorithm behind `rmf_traffic` ?

Conflict avoidance for AGV's in `rmf_traffic` is implemented with time-dependent extension to [A* search](#). This search takes time into account so that it can find paths through space and time that account for the motions of other agents that are in the traffic schedule.

During negotiation, how do fleets compute their proposals?

Because the system is designed to be extensible and adaptable to a wide variety of scenarios and robot vendor combinations, including many that do not currently exist, it has many pieces and hooks for expansion. The sequence of computing a traffic proposal is as follows:

- First, the fleet adapter node will receive a conflict notification which tells it that it needs to participate in a negotiation to resolve a space-time conflict. This notification is received by the `rmf_traffic_ros2::schedule::Negotiation` class.
- For this to happen, the fleet adapter creates a negotiation-notification subscription, so that it will be told whenever a particular robot under its control needs to respond to a negotiation notification.
- When a robot needs to respond to a negotiation, its implementation gets triggered.
- This implementation will launch a multi-threaded `Negotiate` service, whose main implementation can be found [here](#)

Every step in the multi-party negotiation is using the `Negotiate` service the exact same way. The only difference between the various steps is what constraints they need to deal with. Those constraints are described by the `rmf_traffic::schedule::Negotiation::Table::Viewer` object that gets passed to the `respond(~)` function. Because the same object can be used to describe the constraints of all the different blocks in the diagram, we can use the same code to solve every block.

There are also `reject` and `forfeit` code paths that may be invoked as necessary:

- The "rejection" mechanism is used when it's impossible for one of the fleets to accommodate a proposal that came from another. When a rejection is performed, the rejecting fleet will provide a set of feasible trajectories (usually anywhere from 10-200 trajectories) and the fleet that receives the rejection should try once again to find an ideal proposal for itself, but that ideal proposal must accommodate at least one of the trajectory alternatives that were provided with the rejection.
- The "forfeit" mechanism is used when the planner is having an inordinately difficult time finding any kind of solution. This can happen when the negotiation has numerous participants that are all actively on the move, which can lead to situations that are seemingly impossible to resolve due to inconsistencies across time. Typically, when a forfeit is used, there will be another feasible combination of accommodations that gets found by the negotiation. In the worst case scenario, if negotiations keep failing, the robots may experience a real-life deadlock. When a deadlock happens, the participants will be sitting still so the negotiation will reach a steady state and will not be negatively affected by async inconsistencies. When that happens, a successful resolution is practically assured.